

0.51.2. Jusqu'au plus grand : une boucle — doublé, deux positions

Les carrés du A autour de (10, 8), puis ceux du B autour de (9, 9).

1. Ce qu'il faut comprendre

C'est le 0.51 et le 0.51.1 réunis : les carrés du A, puis ceux du B.

Ils se croisent : en passant, les carrés du B effacent le A resté au centre. Chaque lettre ne s'occupe que d'elle-même.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2179 octets** de cartouche.

1. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // 1. Le A, autour de (10, 8)
    for (uint8_t taille = 1; taille <= 8; taille++) {
        carre(10, 8, ALPHABET[0], taille, 1, 50, 1);
    }
    // 2. Le B, autour de (9, 9)
    for (uint8_t taille = 1; taille <= 8; taille++) {
        carre(9, 9, ALPHABET[1], taille, 1, 50, 1);
    }

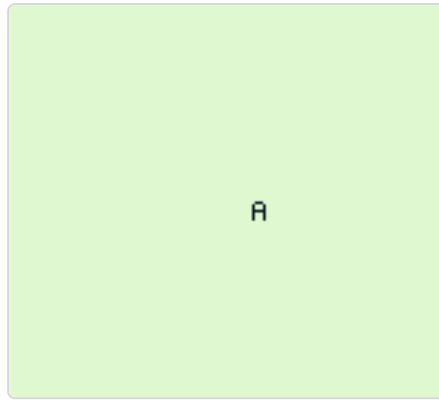
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

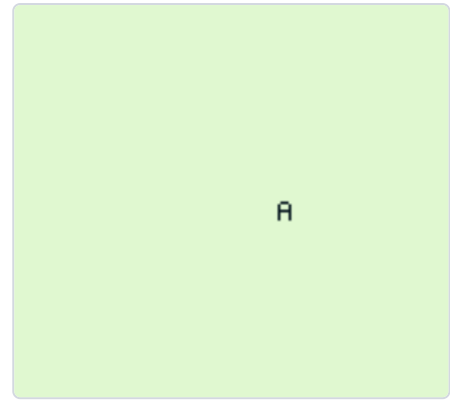
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



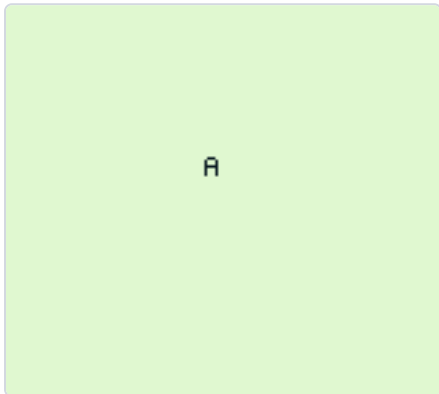
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Les carrés du A, puis ceux du B.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A passe par (12, 10), le coin de son 2e carré

✓ le B finit en (9, 9) (9,9)

Ensuite : **0.52. Deux carrés à la suite** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++